

WEEK 1, LECTURE 1

Applied Industrial Data Science

From Models to Decision Systems

Reproducible modelling, validation, testing, and market-aware prototyping.

INSTRUCTOR

Dr. Muhammad Shoaib

INSTITUTION

GC University Faisalabad

BS Data Science / Analytics

PRACTICE

Senior Data Scientist

Utopi Ltd., United Kingdom

1

THE COURSE QUESTION

Can another person trust and use your work?

This single question guides the whole course.

2

WHY THIS COURSE EXISTS

Beyond the Notebook

Many students can fit a model. Few can build a system another person can trust, rerun, and act on.

The goal: turn a notebook model into a tested, reproducible decision-support prototype.



Industrial reality

Organisations need work that is reproducible, tested, and defensible.



Decision connection

Analysis must inform a real operational or commercial decision.



Engineering practice

Version control, validation, and testing are rarely taught beside modelling.



Career readiness

Internships, projects, and graduate roles expect all of the above.

03

3

SCOPE AND BOUNDARIES

What this course is, and is not



It IS about

- Framing a problem with named users
- Checking data quality, source and origin
- Building a baseline you can justify
- Testing so another person can rerun it
- Stating limits and when not to deploy
- Turning output into a real decision



It is NOT

- A tour of new AI tools
- An advanced ML algorithms course
- A software-engineering or MLOps course
- Causal inference or deployment at scale

Tools appear only where they serve the workflow. The syllabus sets the depth.

04

4

HOW THE WORK CHANGES

The Change in Working Habits

Common Student Habit	Industrial Expectation
One notebook for all code	Structured repo: modules, scripts, config
Hard-coded file paths	Configurable, portable workflows
Manual data cleaning	Repeatable preprocessing and validation
Accuracy-only selection	Robustness, interpretability, and risk
No formal tests	Unit tests and validation checks
Unclear assumptions	Documented limits and non-deployment conditions
Local result on one machine	Reproducible workflow others can rerun
Technical output only	Decision-support tied to user needs

A development path, not a software-engineering syllabus. The course builds these habits gradually.

05

5

THE SEMESTER

One Connected Journey

Each stage depends on the one before it. Value is lost wherever the chain breaks.

06

6

COURSE LEARNING OUTCOMES

Each Stage Proves a Capability

The nine course learning outcomes, mapped onto the journey you have just seen.

1 PROBLEM	2 DATA	3 BUILD	4 CHECK
CLO 1 Formulate A structured task with named users and criteria.	CLO 2 Analyse Quality, Origin, and fitness for modelling.	CLO 3 + 4 Design, Develop A reproducible repo, then a justified baseline.	CLO 5 Diagnose Residuals, segments, robustness, uncertainty.
5 TEST	6 DECIDE	7 PROTOTYPE	8 DEFEND
CLO 6 Implement Tests for what the decision depends on.	CLO 7 Assess Responsible AI, oversight, non-deployment.	CLO 8 Construct A prototype that reaches a stakeholder.	CLO 9 Defend Documentation and oral defence of choices.

Eight stages, nine outcomes. Build earns two, and the ninth binds the rest.

07

7

LEARNING OUTCOME 9

The ninth outcome brings them together

Defend your choices, state your limits, and show how the work supports a decision.

This is the course question, restated as something you can be assessed on.

8

HOW THE COURSE IS ASSESSED

Three Points of Assessment

Weightings are set by the syllabus. The oral defence is mandatory.

20%**Continuous**

Four low-stakes checkpoints: repository setup, data validation, baseline and residual analysis, testing and reflection.

30%**Mid-term**

Project proposal, data understanding, repository structure, baseline pipeline, initial model, validation plan, and an oral review.

50%**Final**

Capstone submission, split 30% technical report, code and prototype, and 20% presentation and oral defence.

Outcome 9 is examined here. The oral defence is where you explain your choices and their limits.

09

9

WEEK 1 FOCUS

From models to decision systems

A model creates value only when it supports a clear, trustworthy decision.

10

THE SHAPE OF THE WORK

The Data-to-Value Pipeline



11

11

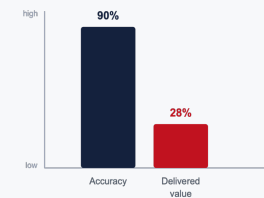
ILLUSTRATIVE EXAMPLE

High Accuracy, Low Value

Academic success

- The code runs.
- The score is high.
- The notebook gives an answer.

... but nobody can act on it.



Illustrative figures, not measured results.

12

12

FRAMING

Start With the Decision, Not the Algorithm

User
Who will act on the result?

Decision
What choice may change?

Evidence
What information is needed to decide?

Consequence
What happens if we are wrong?

13

13

FAILURE MODES

Failure Is Not Only a Modelling Problem

BEFORE modelling

Wrong question, weak source, or unsuitable data.

DURING modelling

Leakage, hidden errors, or ignored uncertainty.

AFTER modelling

No owner, no action, poor timing, or unsafe use.

A single accuracy score cannot reveal every one of these failures.

A break anywhere in the chain destroys the value of everything upstream of it.

14

14

INDUSTRIAL CASE

From Bread Sales to a Responsible Order

The same four questions, on a real order

- **User** the duty manager placing tomorrow's bake order
- **Decision** how many loaves to bake for tomorrow
- **Evidence** sales, stockouts, promotions, weekday, weather
- **Consequence** waste if too many, lost sales if too few



Note: AI-generated FreshMart teaching visual created for this course;

15

15

WHAT TRUST MEANS

Trust Comes From Evidence

not from a polished presentation.

Source
Where did the data come from?

Checks
What evidence shows it works?

Repeatability
Can another person rerun the work?

Limits
When should it not be used?

16

16

EXIT REFLECTION

If the output is wrong, who bears the consequence?

Name the user. Name the decision. Name one risk.

17

WEEK 1 LAB

Lab briefing: tools, workflow, outputs

FreshMart framed the idea. The lab applies it to plant operations.

18

BASELINE TOOLCHAIN

Set Up Your Toolkit This Week



Python 3, pandas, NumPy

The core language and data libraries.



Git and a GitHub account

Version control and a remote to push to.



VS Code

One editor for the notebook, the terminal, and Git.



pytest

For the validation and unit tests to come.

One tool for the whole lab. Add the Python and Jupyter extensions. Record setup errors in your readiness note.

20

19

CORE GIT MECHANICS

Clone Once, Then Repeat

One-time setup

```
git clone https://github.com/drmshoalb/AIDC.git
```

Open the project folder

Read-only until Week 2

Do not clone again

Keep one local project copy

Setup is now complete

Repeated working loop

```
git status
```

edit, then test

```
git add <files>
```

```
git commit -m "clear message"
```

```
git push
```

verify the remote history

21

20

REPOSITORY HYGIENE

Check git status Before Every Commit

A quick check stops passwords, private data, and junk files entering project history.

☑ **Keep**

README, source code, requirements, and small approved teaching data.

🔒 **Exclude with .gitignore**

Passwords and .env, virtual environments, private data, and generated files.

22

21

INDUSTRIAL FOUNDATION

The Course Repository

```
project_root/
├── data/
│   ├── raw/
│   └── processed/
├── src/
│   ├── loading.py
│   ├── model.py
│   ├── test_data.py
│   └── config/
│       ├── settings.yaml
│       ├── README.md
│       └── requirements.txt
```

What each part is for

- **data/** raw and processed, kept apart
- **src/** loading.py, model.py, check_setup.py
- **notebooks/** the Week 1 diagnostic
- **docs/** your draft problem statement
- **tests/, config/, README** checks and setup

Clone it once from github.com/drmshoaib/AIDC
Public and read-only. Your Week 1 commit stays on your machine.

23

22

COMMAND-LINE NAVIGATION

Run the Project

Commands from the project root

- **pwd** confirm the current folder
- **ls** check README.md and src/ are visible
- **python src/check_setup.py** load the dataset
- **python src/model.py** run the baseline
- **python -m pytest -q** run the example checks

Run from the project root. Record the exact command and the error if anything fails.

24

23

UNGRADED · IDENTIFIES GAPS EARLY

Diagnostic Exercise and Readiness

Assessing prerequisites

- Python programming fundamentals
- Data manipulation with pandas
- Introductory statistics and uncertainty
- Basic machine-learning concepts
- Command-line and Git readiness

- 1 Week 1**
Complete the diagnostic task in the lab.
- 2 Week 2**
Review results and identify any gaps.
- 3 Week 3**
Deadline for remediation activities.

No grade penalty. Significant gaps mean a short remediation task before Week 3.

25

24

PRACTICAL OUTPUT

Four Outputs Complete the Week 1 Lab



1 Diagnostic attempt

The prerequisites check. Ungraded, and it only tells us where to help.



2 Labelled plot

Run: load, inspect, and explain a small dataset.



3 Problem statement

Frame: name the user, decision, evidence, and consequence.



4 Commit and readiness note

Record: your first commit. It stays local this week.

The diagnostic is a readiness check, not a graded output. The other three live in your repository. Week 1 commits stay local; you push from Week 2.

26